# OPMIG-06: Processing, Parsing & Transformation

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 6 of 10 | **Created:** December 2025 | **Last Updated:** 10/12/2026

## Table of Contents


1. DQL Processor Commands
2. Dynatrace Pattern Language (DPL)
3. Real-World Log Format Examples ⭐ NEW
4. Common Parsing Patterns
5. Data Transformation Examples
6. Parsing Pattern Library ⭐ NEW
7. ELK/Logstash Migration Patterns ⭐ NEW
8. Technology Bundle Parsers
9. Drop Processors
10. Advanced Processing Patterns
11. Validating Your Processing
12. DPL Architect Tool
13. Complete Processing Pipeline Example


## Learning Objectives

By completing this notebook, you will:

1. Master Dynatrace Pattern Language (DPL) for log parsing
2. Configure DQL processors for data transformation

3. ⭐ **NEW:** Parse Apache, Nginx, and JSON logs with production-ready patterns
4. ⭐ **NEW:** Migrate from ELK/Logstash (Grok to DPL conversion)
5. ⭐ **NEW:** Use the complete Parsing Pattern Library (timestamps, stack traces, HTTP)
6. Implement drop processors for cost optimization
7. Validate parsing success rates and troubleshoot failures

---

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | Dynatrace SaaS with Grail and OpenPipeline access — Managed is not covered by this series |
| **Permissions** | `settings:read` and `settings:write` (OpenPipeline configuration) |
| **API Access** | `logs.read` token scope |
| **DPL Architect** | Access to `https://{env}.apps.dynatrace.com/ui/apps/dynatrace.dpl.architect` |
| **Knowledge** | OPMIG-01 through OPMIG-05; basic regex or pattern matching familiarity |

---

## Processing Stage Overview

Masking, dropping, parsing and transformation are processors in the **Processing** stage — the first stage of every pipeline. The remaining stages (Smartscape node and edge, Permission, Product and Cost allocation, Bucket assignment, Metric extraction, Davis, Data extraction) follow in a fixed order that you cannot change; OPMIG-02 § *Understanding Processing Order* has the full ten-stage table. This notebook covers the Processing stage; OPMIG-07 covers extraction.

### Processor Execution Order

Within the Processing stage, processors execute in the order they're defined, and *"each processor output becomes the input for the next one."* You can reorder them in the UI.

> 💡 **Best Practice:** Order processors logically — mask first, drop second, then parse, then enrich with computed fields based on parsed values.
>
> **Sources:** Processing in OpenPipeline (DT docs) — *"The sequence of stages is fixed for all pipelines and cannot be modified."*

---

# DQL Processor Commands

The DQL processor supports a subset of DQL commands for data transformation.

## Available Commands

| Command | Purpose | Example |
|---|---|---|
| `fieldsAdd` | Add new fields | `fieldsAdd env = "production"` |
| `fieldsRemove` | Remove fields | `fieldsRemove sensitive_field` |
| `fieldsRename` | Rename fields | `fieldsRename old = new_name` |
| `parse` | Extract with DPL | `parse content, "LD? 'count=' INT:count"` |
| `jsonExtract` | Lift a JSON string's root-level keys to fields | JSON logs |
| `fieldsFlatten` | Flatten a nested record into fields | after `parse … "JSON:x"` |
| `fields` / `fieldsKeep` | Keep only the listed fields | |

> **Sources:** DQL commands in OpenPipeline (DT docs) — *"jsonExtract Parses a JSON string and extracts the root-level keys of the JSON object to root-level record fields."*

## fieldsAdd Examples

### Static Values

```
fieldsAdd environment = "production"
| fieldsAdd application = "checkout-service"
| fieldsAdd team = "platform-engineering"
```

### Conditional Values (if/else)

```
fieldsAdd severity = if(loglevel == "ERROR", "critical",
                    else: if(loglevel == "WARN", "warning",
                    else: "info"))
```

### Computed Values

```
fieldsAdd message_length = stringLength(content)
| fieldsAdd short_host = substring(host.name, from: 0, to: 15)
| fieldsAdd is_error = loglevel == "ERROR"
```

### String Operations

```
fieldsAdd normalized_status = lower(status)
| fieldsAdd log_prefix = substring(content, from: 0, to: 50)
| fieldsAdd clean_message = trim(content)
```

### Coalesce (First Non-Null)

```
fieldsAdd effective_level = coalesce(loglevel, status, "UNKNOWN")
```

## fieldsRemove Examples

```
// Remove a single field
fieldsRemove internal_id
```

```
// Remove multiple fields in one command
fieldsRemove temp_field, debug_info, internal_state
```

### fieldsRename Examples

```
// Standardize field names
fieldsRename user_id = userId
| fieldsRename request_id = requestId
| fieldsRename transaction_id = transactionId
```

---

# Dynatrace Pattern Language (DPL)

DPL is a powerful pattern matching language for extracting structured data from text.

## Core Matchers

| Matcher | Description | Matches |
|---|---|---|
| `INT` | Integer | `42`, `-17`, `0` |
| `LONG` | Long integer | `1234567890123` |
| `DOUBLE` | Decimal | `3.14`, `-0.5`, `1.0` |
| `IPADDR` | IP address | `192.168.1.1`, `::1` |
| `IPV4ADDR` | IPv4 only | `10.0.0.1` |
| `IPV6ADDR` | IPv6 only | `2001:db8::1` |
| `LD` | Line data (to delimiter) | Any text until next match |
| `DATA` | Greedy match | Everything remaining |
| `SPACE` | Whitespace | Spaces, tabs |
| `NSPACE` | Non-whitespace | Word-like content |
| `WORD` | Word chars | `hello`, `user123` |
| `JSON` | JSON object | `{"key": "value"}` |
| `EOL` | End of line | Line terminator |

## Pattern Syntax Elements

| Syntax | Meaning | Example |
|---|---|---|
| `MATCHER:field` | Extract to named field | `INT:count` |
| `MATCHER` | Match but don't extract | `SPACE` |
| `MATCHER?` | Optional match | `(':' INT:port)?` |
| `'literal'` | Match exact text | `'error_code='` |
| `(a\|b)` | Alternatives | `('user='\|'userId=')` |
| `MATCHER{n,m}` | Quantifier | `WORD{1,3}` |

> **Quantifier caveat:** `{n,m}` is not universal. `INT` accepts only `*` and `+` — `INT{3}` is rejected. For fixed-width digit runs use a character class: `[0-9]{3}`.

## Basic Parse Examples

```
// Extract user ID after "user=", anywhere in the line
parse content, "LD? 'user=' NSPACE:user_id"

// Extract error code (integer), anywhere in the line
parse content, "LD? 'error_code=' INT:error_code"

// Extract IP and port (line starts with the IP)
parse content, "IPADDR:client_ip ':' INT:port"

// Extract JSON payload
parse content, "LD JSON:payload"
```

> **Three rules that decide whether a pattern returns anything** (each verified with `data record(...)` on 10/02/2026):
> 1. `parse` **anchors at the start of the field.** A pattern that opens with `'user='` returns **null** on every line that does not *begin* with `user=` — no error. Prefix it with `LD?` to skip any leading text (or none). `parse` does *not* need to consume the whole field; trailing text is ignored.
> 2. **A trailing `LD:x` runs to the end of the line**, not the end of the value — `'requestId=' LD:id` on `requestId=abc-123 status=OK` returns `abc-123`

> `status=OK`. End with `NSPACE:x`, a typed matcher, or a closing literal.
> 3. **`LD` never crosses a line break.** In a multi-line record (a stack trace), `LD? 'at ' …` cannot reach line 2. Use `DATA? 'at ' …` for the first match, or `parseAll(content, "…")` to collect every match as an array.
>
> FAQ-15 covers DPL matching semantics in depth.

---

# Real-World Log Format Examples ⭐ NEW

Production-ready DPL patterns for the most common log formats.

## Apache Access Logs (Common Log Format)

**Sample:** `192.168.1.100 - frank [12/Dec/2024:10:30:45 +0000] "GET /api/users HTTP/1.1" 200 1234`

```
parse content, """
  IPADDR:client_ip SPACE '-' SPACE LD:user SPACE
  '[' TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z'):log_time ']' SPACE
  '"' LD:method SPACE LD:request_path SPACE LD:protocol '"' SPACE
  INT:status_code SPACE INT:response_bytes
"""
```

**Extracted:** client_ip, user, log_time, method, request_path, protocol, status_code, response_bytes

## Nginx (with Response Time)

**Sample:** `192.168.1.50 - - [12/Dec/2024:10:30:45 +0000] "POST /api/checkout HTTP/1.1" 201 456 "-" "Mozilla/5.0" "0.342"`

```
parse content, """
  IPADDR:client_ip SPACE '-' SPACE '-' SPACE
  '[' TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z'):log_time ']' SPACE
  '"' LD:method SPACE LD:request_path SPACE LD:protocol '"' SPACE
  INT:status_code SPACE INT:response_bytes SPACE
  '"' LD:referrer '"' SPACE '"' LD:user_agent '"' SPACE
  '"' DOUBLE:response_time_sec '"'
"""
| fieldsAdd response_time_ms = toLong(response_time_sec * 1000)
```

## JSON Application Logs

**Sample:** `{"timestamp":"2024-12-12T10:30:45.123Z","level":"ERROR","service":"payment-api","message":"Payment gateway timeout"}`

**Option 1:** **`jsonExtract`** — lifts the root-level keys of the JSON in `content` to fields (OpenPipeline DQL processor command).

**Option 2: Parse, then pick or flatten**

```
parse content, "JSON:log_data"
| fieldsAdd service = log_data[service]
| fieldsAdd message = log_data[message]
| fieldsAdd level = log_data[level]
```

## Syslog (RFC 3164)

**Sample:** `<34>Dec 12 10:30:45 webserver sshd[1234]: Failed password for admin from 192.168.1.100`

```
parse content, """
  '&lt;' INT:priority '&gt;'
  TIMESTAMP('MMM dd HH:mm:ss'):log_time SPACE
  LD:hostname SPACE LD:app_name '[' INT:pid ']:' SPACE
  DATA:message
"""
// RFC 3164 timestamps carry no year: the parsed value gets the current year
| fieldsAdd facility = toLong(priority / 8)
| fieldsAdd severity = toLong(priority % 8)
```

## Java Application Logs (Log4j)

**Sample:** `2024-12-12 10:30:45,123 [http-nio-8080-exec-5] ERROR com.example.Service - Payment failed`

```
parse content, """
  TIMESTAMP('yyyy-MM-dd HH:mm:ss,SSS'):log_timestamp SPACE
  '[' LD:thread ']' SPACE
  LD:level SPACE LD:logger SPACE '-' SPACE
  DATA:message
"""
```

**Stack Trace Extraction:**

```
parse content, "LD:exception_class ':' SPACE LD:exception_message EOL"
// Frames are on later lines, and LD stops at a line break — DATA? reaches the first frame
| parse content, "DATA? 'at ' LD:error_location '(' LD:file ':' INT:line_number ')'"
// or collect every frame as an array of records
| fieldsAdd frames = parseAll(content, "LD? 'at ' LD:method '(' LD:file ':' INT:line ')'")
```

## Kubernetes / Container Logs

**Sample:** `2024-12-12T10:30:45.123456789Z stdout F {"level":"info","msg":"Request processed"}`

> The `T` separator is written as a plain letter inside `TIMESTAMP('…')` — a quoted `\'T\'` is rejected (*Named pattern element 'T' is not valid*). The parsed value keeps millisecond precision.

```
parse content, """
  TIMESTAMP('yyyy-MM-ddTHH:mm:ss.SSSSSSSSSZ'):k8s_timestamp SPACE
  LD:stream SPACE LD:log_tag SPACE
  JSON:log_payload
"""
| fieldsAdd level = log_payload[level]
| fieldsAdd msg = log_payload[msg]
```

---

# Common Parsing Patterns

## Apache/Nginx Access Logs

**Sample log:**

```
192.168.1.100 - - [12/Dec/2024:10:30:45 +0000] "GET /api/users HTTP/1.1" 200
1234
```

**DPL Pattern:**

```
parse content, "IPADDR:client_ip SPACE '-' SPACE LD:user SPACE '[' LD:log_time
']' SPACE '\"' LD:method SPACE LD:path SPACE LD:protocol '\"' SPACE
INT:status_code SPACE INT:bytes"
```

## Key-Value Logs

**Sample log:**

```
userId=12345, action=login, status=success, duration=150ms
```

**DPL Patterns:**

```
// Extract each key-value pair, wherever it sits in the line
parse content, "LD? 'userId=' INT:user_id"
| parse content, "LD? 'action=' LD:action ','"
| parse content, "LD? 'status=' LD:status ','"
| parse content, "LD? 'duration=' INT:duration_ms 'ms'"
```

## Flexible User ID Extraction

**Sample logs with varying formats:**

```
Processing request for user=john123
User userId=john123 authenticated
Request from user_id=john123 received
```

**DPL Pattern (alternatives):**

```
parse content, "LD? ('user='|'userId='|'user_id=') NSPACE:user_id"
```

### Timestamp Parsing

**DPL with timestamp format:**

```
parse content, "TIMESTAMP('yyyy-MM-dd HH:mm:ss'):log_timestamp"
parse content, "TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z'):apache_time"
```

### Optional Fields

**Sample log:**

```
Request to server:8080 completed
Request to server completed
```

**DPL Pattern (optional port):**

```
parse content, "'Request to ' LD:server (':' INT:port)? ' completed'"
```

### Stack Trace Extraction

**DPL Pattern:**

```
parse content, "LD:exception_class ':' SPACE LD:exception_message"
// first line of the record only — LD stops at the line break
```

---

## Data Transformation Examples

### Example 1: Parse and Enrich Application Logs

**Input:**

```
{"content": "[2024-12-12T10:30:45] ERROR PaymentService - Payment failed for
orderId=12345, amount=99.99"}
```

**Processor 1: Parse log structure**

```
parse content, "'[' TIMESTAMP('yyyy-MM-ddTHH:mm:ss'):log_time ']' SPACE
LD:level SPACE LD:service ' - ' DATA:message"
```

**Processor 2: Extract payment details**

```
parse content, "LD? 'orderId=' INT:order_id"
| parse content, "LD? 'amount=' DOUBLE:amount"
```

**Processor 3: Add computed fields**

```
fieldsAdd severity = if(level == "ERROR", "critical", else: "normal")
| fieldsAdd application_tier = "payment"
```

## Example 2: JSON Log Processing

**Input:**

```
{"content": "{\"level\":\"info\",\"msg\":\"Request
processed\",\"duration_ms\":150}"}
```

**Processor: Parse JSON and flatten**

```
parse content, "JSON:json_data"
```

To lift the keys to top-level fields, follow with `fieldsFlatten json_data` — or use `jsonExtract` instead of `parse`.

## Example 3: Multi-Format Log Normalization

**Processor: Normalize different log level formats**

```
fieldsAdd normalized_level = if(contains(content, "ERROR") OR
contains(content, "error"), "ERROR",
                            else: if(contains(content, "WARN") OR
contains(content, "warn"), "WARN",
                            else: if(contains(content, "INFO") OR
contains(content, "info"), "INFO",
                            else: if(contains(content, "DEBUG") OR
contains(content, "debug"), "DEBUG",
                            else: "UNKNOWN"))))
```

---

# Parsing Pattern Library ⭐ NEW

## Timestamp Patterns (10+ Formats)

| Format | Example | DPL |
|---|---|---|
| ISO 8601 | `2024-12-12T10:30:45Z` | `TIMESTAMP('yyyy-MM-ddTHH:mm:ssZ')` |
| ISO + MS | `2024-12-12T10:30:45.123Z` | `TIMESTAMP('yyyy-MM-ddTHH:mm:ss.SSSZ')` |
| Apache | `12/Dec/2024:10:30:45 +0000` | `TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z')` |
| Syslog | `Dec 12 10:30:45` | `TIMESTAMP('MMM dd HH:mm:ss')` |
| Java | `2024-12-12 10:30:45,123` | `TIMESTAMP('yyyy-MM-dd HH:mm:ss,SSS')` |
| MySQL | `2024-12-12 10:30:45` | `TIMESTAMP('yyyy-MM-dd HH:mm:ss')` |
| Unix Epoch | `1702380645` | `LONG:epoch` → `toTimestamp(epoch * 1000)` |

## HTTP Request Patterns

```
// Full request line (a literal " inside the DQL string is escaped as \")
parse content, "LD? '\"' LD:method SPACE LD:path SPACE LD:protocol '\"'"

// Separate path and query
parse content, "LD? '\"' LD:method SPACE LD:path ('?' LD:query)? SPACE
LD:protocol '\"'"

// RESTful API paths (/api/v1/users/12345)
parse content, "LD? '/api/v' INT:api_version '/' LD:resource '/' INT:id"
```

## Key-Value Patterns

```
// Simple: user=john count=42
parse content, "'user=' LD:user SPACE 'count=' INT:count"

// Quoted: user="John Doe" email="john@example.com"
parse content, "'user=\"' LD:user '\"' SPACE 'email=\"' LD:email '\"'"

// Logfmt: level=info msg="OK" duration=150ms
parse content, "'level=' LD:level SPACE 'msg=\"' LD:msg '\"' SPACE 'duration='
INT:dur 'ms'"
```

## Stack Trace Patterns

```
// Java exception (first line)
parse content, "LD:exception_class ':' SPACE LD:exception_message EOL"

// Stack trace lines — every frame, as an array (LD cannot cross a line break)
fieldsAdd frames = parseAll(content, "LD? 'at ' LD:class_method '(' LD:file
':' INT:line ')'")

// Caused by — anywhere in the record
fieldsAdd causes = parseAll(content, "'Caused by: ' LD:caused_by ':' SPACE
LD:message")

// Python traceback — first frame
parse content, "DATA? 'File \"' LD:file '\", line ' INT:line ', in '
NSPACE:function"
```

## PII Masking Patterns

Masking uses `replacePattern`, which takes a **DPL** pattern (not a regular expression — there is no `replaceAll` function). See OPMIG-08 for the full masking set.

```
// Credit cards (4111-1111-1111-1111 → ****-****-****-****); bounded so IDs
and UUIDs are left alone
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "****-****-****-****")

// Email addresses
| fieldsAdd content = replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-
9.-]+", replacement: "***@***.***")

// SSN (123-45-6789 → ***-**-****)
| fieldsAdd content = replacePattern(content, "[0-9]{3} '-' [0-9]{2} '-' [0-9]
{4}", replacement: "***-**-****")
```

**Partial masking ("keep the last 4 digits")** needs a lookahead, not a back-reference. `replacePattern` has no back-references, so the old `$4` example cannot be expressed, but a `>>` lookahead checks the last four digits without consuming them, so they survive the replacement. Bound both ends so the pattern cannot fire inside a longer number or an ID:

```
// 4111-1111-1111-1234 → ****-****-****-1234
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] [0-9]{4} '-
' [0-9]{4} '-' [0-9]{4} '-' &gt;&gt;([0-9]{4} !&gt;&gt;[0-9A-Za-z-])",
replacement: "****-****-****-")
// 4111111111111234 → XXXXXXXXXXXX1234 (unbroken; the letter bound keeps hex
IDs such as 3f2b9c1e4111111111111111aa intact)
| fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] [0-9]{12}
&gt;&gt;([0-9]{4} !&gt;&gt;[0-9A-Za-z-])", replacement: "XXXXXXXXXXXX")
// 123-45-6789 → ***-**-6789
| fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] [0-9]{3}
'-' [0-9]{2} '-' &gt;&gt;([0-9]{4} !&gt;&gt;[0-9A-Za-z-])", replacement: "***-
**-")
```

All three statements were run with `data record(...)` on 10/12/2026 and returned the values in the comments; `card=4111-1111-1111-1111, cvv=123` became `card=****-****-****-1111, cvv=123`.

### Network Patterns

```
// IP and port, anywhere in the line
parse content, "LD? IPADDR:ip ':' INT:port"

// IPv4 only
parse content, "LD? IPV4ADDR:ipv4"

// IPv6 only
parse content, "LD? IPV6ADDR:ipv6"

// URL parsing (an LD must be followed by something it can stop at, so the
path starts at '/')
parse content, "LD:protocol '://' LD:hostname (':' INT:port)? '/' LD:path"
```

---

---

## ELK/Logstash Migration Patterns ⭐ NEW

### Grok vs. DPL: Key Differences

| Grok (Logstash) | DPL (OpenPipeline) |
|---|---|
| `%{IP:client_ip}` | `IPADDR:client_ip` |
| `%{INT:count}` | `INT:count` |
| `%{WORD:user}` | `WORD:user` |
| `%{GREEDYDATA:msg}` | `DATA:msg` |
| `\\s+` | `SPACE` |

### Apache Log Migration

**Grok:** `%{COMBINEDAPACHELOG}`

**DPL:**

```
parse content, """
  IPADDR:client_ip SPACE LD:ident SPACE LD:auth SPACE
  '[' TIMESTAMP('dd/MMM/yyyy:HH:mm:ss Z'):log_time ']' SPACE
  '"' LD:method SPACE LD:request_path (SPACE LD:http_version)? '"' SPACE
  INT:status_code SPACE (INT:response_bytes | '-') SPACE
  '"' LD:referrer '"' SPACE '"' LD:user_agent '"'
"""
```

## Logstash Filter → OpenPipeline Mapping

| Logstash Filter | OpenPipeline |
|---|---|
| `grok { }` | DQL parse processor |
| `mutate { add_field }` | `fieldsAdd field = "value"` |
| `mutate { remove_field }` | `fieldsRemove field` |
| `mutate { rename }` | `fieldsRename old = new` |
| `drop { }` | Drop processor |
| `json { }` | `jsonExtract`, or `parse content, "JSON:x"` + `fieldsFlatten x` |
| `date { }` | TIMESTAMP in parse |

## Complete Migration Example

**Logstash:**

```
filter {
  if [level] == "DEBUG" { drop { } }
  grok { match =&gt; { "message" =&gt; "%{TIMESTAMP_ISO8601:ts} %{LOGLEVEL:level} %{GREEDYDATA:msg}" } }
  mutate { add_field =&gt; { "env" =&gt; "production" } }
}
```

**OpenPipeline:**

1. Drop processor: `loglevel == "DEBUG"`
2. DQL parse: `parse content, "TIMESTAMP('yyyy-MM-dd HH:mm:ss'):ts SPACE LD:level`

`SPACE DATA:msg"`
3. DQL add: `fieldsAdd env = "production"`

---

## Technology Bundle Parsers

The **Technology bundle** processor applies a library of predefined processors for a selected technology. The documented library is grouped by category, for example:

| Category | Examples from the library |
|---|---|
| Servers | Apache HTTP (access, error logs), Nginx (access, error log), Apache Tomcat, HAProxy |
| Syslog | RFC 3164, RFC 5424, Generic, Cisco switch |
| Applications | Java (Spring Boot, Slf4j, Log4j, Logback, Util Logging, exception stack trace), Node.js, Go |
| AWS / Azure | API Gateway, CloudFront, EKS control plane, AKS, and more |

There is no generic "JSON" bundle — for JSON logs use `jsonExtract` or `parse … "JSON:x"` in a DQL processor. The fields a bundle adds are defined by the bundle; check its output on a sample record before you rely on a field name.

> **Sources:** Technology bundle processors for logs (DT docs) — *"The following table lists the technology bundles available in OpenPipeline."* (library read 10/02/2026).

### When to Use Technology Parsers

| Use Technology Parser | Use Custom DQL/DPL |
|---|---|
| Standard log format | Custom/proprietary format |
| Quick setup needed | Specific field extraction |
| Common technology | Conditional parsing |

| Use Technology Parser | Use Custom DQL/DPL |
|---|---|
| All fields needed | Selective extraction |

### Configuring Technology Parsers

1. Open pipeline in OpenPipeline settings
2. Go to the **Processing** stage
3. Click **+ Processor → Technology bundle**
4. Select the technology
5. Configure matching condition
6. Save

---

## Drop Processors

Drop processors remove records from the pipeline before storage.

### Common Drop Patterns

| Use Case | Matching Condition |
|---|---|
| Debug logs | `loglevel == "DEBUG"` |
| Trace logs | `loglevel == "TRACE"` |
| Health checks | `matchesValue(content, "*health*")` |
| Readiness probes | `matchesValue(content, "*/ready*")` |
| Metrics endpoints | `matchesValue(content, "*/metrics*")` |
| Heartbeats | `matchesPhrase(content, "heartbeat")` |

> Matching conditions accept `matchesValue` (with `*` wildcards, any substring) and `matchesPhrase` (whole tokens); `contains()` is not enabled in a matcher.
> | Specific source | `log.source == "noisy-service"` |

### Drop Processor Configuration

```
Processor Type: Drop
Name: Drop debug logs
Matching Condition: loglevel == "DEBUG" OR status == "DEBUG"
```

### Combining Drop Conditions

```
// Drop all non-essential logs
loglevel == "DEBUG"
  OR loglevel == "TRACE"
  OR matchesValue(content, "*health*")
  OR matchesValue(content, "*/metrics*")
```

> ⚠️ **Important:** Dropped data is gone forever. Test drop conditions carefully before deploying.

---

## Advanced Processing Patterns

### Pattern 1: Fix Missing Timestamp

When logs have a custom timestamp format that's not recognized:

```
// Parse timestamp from content
parse content, "'[' TIMESTAMP('yyyy-MM-dd HH:mm:ss'):parsed_timestamp ']'"
```

Parse into a **new** field (`parsed_timestamp`, `log_time`). Overwriting `timestamp` risks the record failing log schema validation — a stored log must have a `timestamp` *"Within the ingestion range"*, and one that is not is dropped. Set `timestamp` deliberately and only from a parsed `TIMESTAMP` value.

> **Sources:** OpenPipeline limits (DT docs) — *Schema validation for logs*: *"A processed log record is persisted only if all the following field conditions are satisfied. If the schema is not valid, the log is*

> *dropped."*

## Pattern 2: Fix Missing Log Level

When logs don't have a standard loglevel field:

```
// Extract level from content
fieldsAdd loglevel = if(contains(content, "[ERROR]") OR contains(content,
"ERROR:"), "ERROR",
                    else: if(contains(content, "[WARN]") OR contains(content,
"WARN:"), "WARN",
                    else: if(contains(content, "[INFO]") OR contains(content,
"INFO:"), "INFO",
                    else: if(contains(content, "[DEBUG]") OR
contains(content, "DEBUG:"), "DEBUG",
                    else: "NONE"))))
```

## Pattern 3: Parse JSON from Within Text

When JSON is embedded in a log message:

```
// Extract JSON from text (trailing text after the JSON is ignored)
parse content, "LD JSON:embedded_json"
```

## Pattern 4: Compute Response Time Categories

```
fieldsAdd response_category = if(duration_ms &lt; 100, "fast",
                              else: if(duration_ms &lt; 500, "normal",
                              else: if(duration_ms &lt; 1000, "slow",
                              else: "very_slow")))
```

## Pattern 5: Standardize Boolean Fields

```
fieldsAdd is_success = if(status == "success" OR status == "ok" OR status ==
"200", true, else: false)
```

### Pattern 6: Extract Service Name from Path

```
// From "GET /api/v1/users HTTP/1.1" → users
parse content, "LD? '/api/v' INT '/' NSPACE:service_name"
```

---

## Validating Your Processing

After configuring processors, validate that parsing is working correctly.

```
// Check parsing success rate across all pipelines
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    with_loglevel = countIf(isNotNull(loglevel)),
    with_status = countIf(isNotNull(status)),
    with_either = countIf(isNotNull(loglevel) OR isNotNull(status))
  }
| fieldsAdd parsing_rate = round((toDouble(with_either) / toDouble(total)) *
100, decimals: 1)
```

```
// Parsing success rate by pipeline
fetch logs, from: now() - 1h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {
    total = count(),
    parsed = countIf(isNotNull(loglevel))
  }, by: {dt.openpipeline.pipelines}
| fieldsAdd parsing_rate = round((toDouble(parsed) / toDouble(total)) * 100,
decimals: 1)
| sort parsing_rate asc
```

```
// Sample logs that failed parsing (no loglevel extracted)
fetch logs, from: now() - 1h
| filter isNull(loglevel) AND isNull(status)
| fields timestamp, log.source, dt.openpipeline.pipelines, content
| limit 25
```

```
// Verify specific parsed fields exist
// Replace 'your_field' with fields your parsing should create
fetch logs, from: now() - 1h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {
    total = count(),
    with_user_id = countIf(isNotNull(user_id)),
    with_request_id = countIf(isNotNull(request_id))
  }, by: {dt.openpipeline.pipelines}
```

```
// Sample successfully parsed logs to verify field extraction
fetch logs, from: now() - 1h
| filter isNotNull(loglevel)
| limit 20
```

```
// Verify drop processors are working
// If drops are configured, debug log count should be low
fetch logs, from: now() - 1h
| summarize {debug_count = countIf(loglevel == "DEBUG")}
| fieldsAdd message = if(debug_count == 0, "✅ Drop processor working - no
debug logs",
                       else: "⚠️ Debug logs still present - check drop
configuration")
```

```
// Check log level distribution after processing
fetch logs, from: now() - 1h
| summarize {log_count = count()}, by: {loglevel}
| sort log_count desc
```

---

# DPL Architect Tool

Dynatrace provides a **DPL Architect** tool for building and testing patterns:

## Accessing DPL Architect

1. Navigate to **Settings** → **OpenPipeline**
2. When adding a parse processor, click **Open DPL Architect**

3. Or access via: `https://{your-environment}.apps.dynatrace.com/ui/apps/dynatrace.dpl.architect`

### Using DPL Architect

1. Paste sample log content
2. Build pattern interactively
3. See extracted fields in real-time
4. Copy pattern to processor definition

> 💡 **Tip:** Always test patterns in DPL Architect before deploying to production pipelines.

---

## Complete Processing Pipeline Example

### Pipeline: `application-logs`

**Processor 1: Drop Debug (Drop)**

```
Matching: loglevel == "DEBUG" OR matchesValue(content, "*[DEBUG]*")
```

**Processor 2: Parse Application Log (DQL)**

```
parse content, "'[' TIMESTAMP('yyyy-MM-dd HH:mm:ss'):log_ts ']' SPACE '['
LD:level ']' SPACE '[' LD:thread ']' SPACE LD:class ' - ' DATA:message"
```

**Processor 3: Extract Request ID (DQL)**

```
parse content, "LD? 'requestId=' NSPACE:request_id"
```

**Processor 4: Add Environment Tags (DQL)**

```
fieldsAdd environment = "production"
| fieldsAdd application = "checkout-service"
| fieldsAdd team = "platform"
```

**Processor 5: Compute Severity (DQL)**

```
fieldsAdd severity = if(level == "ERROR", "P1",
                     else: if(level == "WARN", "P2",
                     else: "P3"))
```

---

## Next Steps

Now that you can transform data, continue with:

| Notebook | Focus Area |
|---|---|
| **OPMIG-07** | Metric & Event Extraction |
| **OPMIG-08** | Security, Masking & Compliance |
| **OPMIG-09** | Troubleshooting & Validation |

---

## References

- OpenPipeline Processing
- Processing Examples
- Dynatrace Pattern Language
- DPL Architect Tool
- DQL Functions in OpenPipeline

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*